

CI JENKINS

CI/CD MASTERY COURSE

From Zero to Production-Ready Pipelines

A Complete Hands-On Curriculum by a 20+ Year DevOps Veteran

9 Topics	6 Projects	50+ Exercises
Structured Learning	Resume Projects	Practice Questions

Table of Contents

STUDY PLAN	How to Study This Course — Week by Week
PREREQ	Prerequisites & Self-Assessment Questions
TOPIC 1	CI/CD Fundamentals & Core Concepts
TOPIC 2	Installing & Configuring Jenkins
TOPIC 3	Jenkins Jobs — Freestyle & Pipeline
TOPIC 4	Jenkinsfile & Declarative Pipeline Syntax
TOPIC 5	Source Control Integration (Git & GitHub)
TOPIC 6	Build Tools — Maven, Gradle, npm
TOPIC 7	Testing, Code Quality & SonarQube
TOPIC 8	Docker Integration in Jenkins Pipelines
TOPIC 9	Deployment Strategies & Best Practices
PROJECT 1	Personal Portfolio Auto-Deploy Pipeline
PROJECT 2	Java Spring Boot CI/CD — Full Pipeline
PROJECT 3	Node.js App with Docker & Registry Push
PROJECT 4	Multi-Branch Pipeline with Feature Flags
PROJECT 5	Blue/Green Deployment Pipeline
PROJECT 6	End-to-End DevOps Pipeline (Resume Star)
APPENDIX	Cheat Sheet, Commands & Quick Reference

Study Plan — How to Learn This Course

This course is designed for **6 weeks of structured learning**. Each week builds on the last. Follow this plan consistently and you will go from beginner to job-ready in 6 weeks.

Week 1

Foundation & Setup

- Study Topic 1: CI/CD Fundamentals (Days 1-2)
- Study Topic 2: Install & Configure Jenkins (Days 3-4)
- Complete all Topic 1 & 2 practice questions (Day 5)
- Weekend: Start Project 1 — Personal Portfolio Pipeline

Week 2

Jobs & Pipelines

- Study Topic 3: Jenkins Jobs — Freestyle & Pipeline (Days 1-2)
- Study Topic 4: Jenkinsfile & Declarative Syntax (Days 3-4)
- Practice writing 5 different Jenkinsfiles from scratch (Day 5)
- Weekend: Complete Project 1, Start Project 2

Week 3

SCM & Build Tools

- Study Topic 5: Git & GitHub Integration (Days 1-2)
- Study Topic 6: Build Tools — Maven, Gradle, npm (Days 3-4)
- Set up webhook triggers on a real GitHub repo (Day 5)
- Weekend: Complete Project 2 — Java Spring Boot Pipeline

Week 4

Quality & Testing

- Study Topic 7: Testing, Code Quality & SonarQube (Days 1-3)
- Integrate SonarQube with a real project (Day 4)
- Review and redo all practice questions so far (Day 5)
- Weekend: Start Project 3 — Node.js with Docker

Week 5**Docker & Deployment**

- Study Topic 8: Docker Integration in Jenkins (Days 1-2)
- Study Topic 9: Deployment Strategies (Days 3-4)
- Build a full Docker build-push-deploy pipeline (Day 5)
- Weekend: Projects 4 & 5 — Multi-branch & Blue/Green

Week 6**Capstone & Resume**

- Review all topics — focus on weak areas (Days 1-2)
- Build Project 6: Full End-to-End Pipeline (Days 3-4)
- Document all projects on GitHub with README files (Day 5)
- Weekend: Polish resume, prepare portfolio, mock interview practice

PRO TIP

Consistency over intensity. 2 focused hours per day beats 10 hours on a weekend. Set a fixed daily time slot for this course and protect it. Every topic has hands-on exercises — never skip them.

Prerequisites & Self-Assessment

Before diving into Jenkins, you need a baseline in a few areas. This section checks your existing knowledge and tells you what to review if needed.

Linux & Command Line

Practice Questions & Answers

Q1. What does the command 'chmod +x script.sh' do?

Answer: *Makes the file script.sh executable by adding execute permission.*

Q2. How do you check which process is using port 8080?

Answer: *Use: lsof -i :8080 or netstat -tulnp | grep 8080*

Q3. What is the difference between a relative and absolute path?

Answer: *Absolute path starts from root (/home/user), relative path is from current directory (../folder).*

Q4. How do you search for text inside files recursively?

Answer: *Use: grep -r 'search-term' /path/to/dir*

Q5. What does 'export MY_VAR=hello' do in bash?

Answer: *Creates an environment variable MY_VAR with value 'hello' available to child processes.*

Git & Version Control

Practice Questions & Answers

Q1. What is the difference between git merge and git rebase?

Answer: Merge combines branches with a merge commit. Rebase replays commits on top of another branch for a linear history.

Q2. What does 'git fetch' do vs 'git pull'?

Answer: fetch downloads remote changes but does NOT merge. pull = fetch + merge.

Q3. How do you undo the last commit without losing changes?

Answer: Use: git reset --soft HEAD~1 (keeps changes staged)

Q4. What is a webhook in GitHub?

Answer: A webhook sends an HTTP POST to a URL whenever a specific event (like a push) happens in a repo.

Q5. What is the purpose of a .gitignore file?

Answer: It tells Git which files and directories to NOT track (e.g. node_modules/, .env files, build outputs).

Basic Networking

Practice Questions & Answers

Q1. What is the difference between HTTP and HTTPS?

Answer: HTTPS is HTTP with TLS/SSL encryption. Data is encrypted in transit.

Q2. What does a reverse proxy do?

Answer: It sits in front of servers, forwards client requests to the right backend, and can handle SSL, load balancing, caching.

Q3. What is port 443 commonly used for?

Answer: HTTPS traffic.

Q4. Explain what DNS does in simple terms.

Answer: DNS translates human-readable domain names (google.com) into IP addresses (142.250.x.x) computers use.

Docker Basics

Practice Questions & Answers

Q1. What is the difference between a Docker image and a container?

Answer: *An image is a read-only blueprint. A container is a running instance of an image.*

Q2. What does 'docker build -t myapp:1.0 .' do?

Answer: *Builds a Docker image named myapp with tag 1.0 using the Dockerfile in the current directory.*

Q3. How do you see all running containers?

Answer: *Use: docker ps*

Q4. What is a Docker volume used for?

Answer: *Volumes persist data outside the container lifecycle so data survives container restarts/removal.*

INFO

How to use this section: Go through every question honestly. If you got less than 3/5 in any area — spend 2-3 days reviewing that topic on YouTube or in documentation BEFORE starting Topic 1. A weak foundation will make everything harder.

1

CI/CD Fundamentals & Core Concepts

Understanding the WHY before the HOW

1.1 What is CI/CD?

Continuous Integration (CI) is the practice of automatically building and testing code every time a developer pushes a change. The goal: catch bugs early, before they reach production.

Continuous Delivery (CD) extends CI by automatically preparing code for release to a staging or production environment. The deployment still requires a human approval.

Continuous Deployment goes one step further — every change that passes all tests is automatically deployed to production with zero human intervention.

Term	Definition	Human Approval?
Continuous Integration	Auto build + test on every commit	Not required
Continuous Delivery	Auto build + test + stage for release	Required to deploy
Continuous Deployment	Auto build + test + deploy to prod	Not required

1.2 The CI/CD Pipeline — Stages Explained

A pipeline is a sequence of automated steps that take your code from commit to production. Here is the typical flow:

```
# Typical CI/CD Pipeline Flow

Developer pushes code

|

[Stage 1: SOURCE] — Checkout code from Git

|

[Stage 2: BUILD] — Compile code (Maven/Gradle/npm)
```



```
|  
  
[Stage 3: TEST] – Run unit & integration tests  
  
|  
  
[Stage 4: ANALYZE] – Code quality scan (SonarQube)  
  
|  
  
[Stage 5: PACKAGE] – Build Docker image / artifact  
  
|  
  
[Stage 6: PUBLISH] – Push to registry / artifact store  
  
|  
  
[Stage 7: DEPLOY] – Deploy to staging / production  
  
|  
  
[Stage 8: NOTIFY] – Send Slack/email notification
```

1.3 Why Jenkins? — The Market Leader

Feature	Jenkins	GitHub Actions	GitLab CI	CircleCI
Open Source	Yes (free)	Free tier	Free tier	Free tier
Self-hosted	Yes	Runners only	Yes	Yes
Plugin ecosystem	1800+ plugins	Limited	Good	Moderate
Enterprise use	Very high	Growing	High	Moderate
Groovy scripting	Full support	No	No	No
Job market demand	#1 in listings	#2	#3	Lower

1.4 Key Jenkins Terminology

Term	What It Means
Job / Project	The basic unit of work in Jenkins. A configured task to run.
Pipeline	A series of jobs/stages defined as code (Jenkinsfile).

Build	A single execution of a job.
Stage	A logical phase in a pipeline (Build, Test, Deploy).
Step	A single command or action inside a stage.
Agent / Node	The machine where Jenkins runs the build.
Master / Controller	The main Jenkins server that manages jobs and agents.
Workspace	The directory on the agent where Jenkins checks out code.
Artifact	Output of a build — a JAR, Docker image, ZIP file, etc.
Trigger	What starts a build — push, schedule, manual, etc.
Plugin	An extension that adds functionality to Jenkins.
Jenkinsfile	A text file that defines a pipeline as code, stored in SCM.

1.5 The Software Delivery Lifecycle (SDLC) in Context

Jenkins sits at the heart of the modern SDLC. Understanding where CI/CD fits in helps you design better pipelines:

```
Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor
```

```
^ |
```

```
|_____ Feedback Loop (Jenkins notifies) _____|
```

Jenkins automates: Build, Test, Release, and Deploy stages.

DevOps culture: short feedback loops, frequent small releases.

Topic 1 — Practice Questions

Practice Questions & Answers

Q1. What is the key difference between Continuous Delivery and Continuous Deployment?

Answer: *In Continuous Delivery, a human must approve the final deployment. In Continuous Deployment, every passing build is deployed automatically with no human intervention.*

Q2. Name 3 stages that every good CI/CD pipeline should have.

Answer: *At minimum: Source (checkout), Build (compile), Test (run tests). Best practice adds: Analyze (SonarQube), Package (Docker), Deploy (to environment), Notify (Slack/email).*

Q3. Why is it better to catch bugs in CI rather than production?

Answer: *It's 10-100x cheaper to fix a bug in dev/CI than in production. Production bugs cause downtime, data loss, and customer impact. CI gives immediate feedback to developers while context is fresh.*

Q4. What is a Jenkinsfile and why is storing it in Git important?

Answer: *A Jenkinsfile is a text file that defines your pipeline as code. Storing it in Git means your pipeline is version-controlled, reviewable, auditable, and reproducible — just like application code.*

Q5. A developer asks: 'Why not just run tests manually before pushing?' How do you respond?

Answer: *Humans forget. Manual testing doesn't scale with team size. CI ensures every commit is tested consistently, in a clean environment, by the same automated process — eliminating human error and creating an audit trail.*

Q6. What is meant by 'fail fast' in CI/CD?

Answer: *Fail fast means the pipeline stops and notifies developers as soon as any step fails, rather than running all steps. This saves time and resources by not continuing to build/test/deploy broken code.*

2

Installing & Configuring Jenkins

Getting your Jenkins server ready for work

2.1 Jenkins Installation Options

Method	Best For	Command/Notes
Docker (recommended)	Quick start, any OS	<code>docker run -p 8080:8080 jenkins/jenkins:lts</code>
WAR file	Custom Java setups	<code>java -jar jenkins.war</code>
Linux package	Production servers	<code>apt-get install jenkins</code>
Kubernetes Helm	Enterprise scale	<code>helm install jenkins jenkinsci/jenkins</code>

2.2 Installing Jenkins with Docker — Step by Step

```
# Docker Install

# Step 1: Create a Docker network for Jenkins
docker network create jenkins

# Step 2: Run Jenkins with persistent volume
docker run -d \
  --name jenkins \
  --network jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins-data:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts-jdk17

# Step 3: Get the initial admin password
```

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

```
# Step 4: Open browser -> http://localhost:8080
```

```
# Paste the password and follow the setup wizard
```

```
# Step 5: Install 'suggested plugins' when prompted
```

```
# Step 6: Create your first admin user
```

INFO

Port 8080 is the Jenkins web UI. Port 50000 is used for Jenkins agents (build workers) to connect to the controller. Always mount the Docker socket if you plan to run Docker commands inside your pipelines.

2.3 Essential Plugins to Install

Plugin Name	Purpose	Priority
Pipeline	Core pipeline/Jenkinsfile support	Required
Git	Clone from Git repositories	Required
GitHub Integration	Webhooks, PR status checks	Required
Docker Pipeline	<code>docker.build()</code> , <code>docker.image()</code> in pipeline	Required
Blue Ocean	Modern UI for pipeline visualization	Highly Recommended
SonarQube Scanner	Code quality analysis integration	Recommended
Slack Notification	Send build notifications to Slack	Recommended
Email Extension	Rich email notifications	Recommended
Credentials Binding	Inject secrets safely into builds	Required
Pipeline Stage View	View stages in classic UI	Recommended
Maven Integration	Maven build support	For Java projects
NodeJS	Node.js tool management	For Node projects
Kubernetes	Run builds on Kubernetes pods	Advanced
Blue Ocean	Beautiful pipeline visualization	Recommended

2.4 Jenkins Configuration — Global Settings

After installation, configure these in **Manage Jenkins > Configure System**:

```
# Post-Install Configuration

1. JENKINS_URL

Set to: http://your-server-ip:8080/

Why: Used in notification links, webhook callbacks

2. System Admin Email Address

Set to: jenkins@yourcompany.com

Why: From address on email notifications

3. Git Global Config (Manage Jenkins > Global Tool Config)

Name: Default

Path: git (or /usr/bin/git)

4. JDK Installation

Add JDK 17 (or the version your projects use)

Either install automatically OR point to existing path

5. Maven Installation

Name: Maven-3.9

Install from Apache automatically

6. Node.js Installation

Name: NodeJS-18

Install from nodejs.org automatically
```

2.5 Security Configuration — Never Skip This

COMMON MISTAKE

Critical: Jenkins is a powerful server. Exposed to the internet without proper security = instant compromise. Always complete these steps before using Jenkins.

Security Checklist:

- [] Enable security: Manage Jenkins > Configure Global Security
- [] Set authorization to: Matrix-based security
- [] Only logged-in users can do anything
- [] Create individual user accounts (never share admin)
- [] Enable CSRF protection (on by default in modern Jenkins)
- [] If internet-facing: put Nginx reverse proxy with SSL in front
- [] Disable CLI via HTTP (attack vector if not used)
- [] Set up a reverse proxy with SSL certificate
- [] Regularly update Jenkins and all plugins

Topic 2 — Practice Questions

Practice Questions & Answers

Q1. What is the purpose of mounting `/var/run/docker.sock` into the Jenkins container?

Answer: *It allows Jenkins to communicate with the host Docker daemon, so pipeline steps can run `docker build`, `docker push`, etc. directly from inside the Jenkins container.*

Q2. What is stored in `/var/jenkins_home`?

Answer: *All Jenkins configuration, job definitions, build history, plugin data, credentials, and workspace files. This is why mounting it as a volume is critical — otherwise all data is lost when the container restarts.*

Q3. What is the difference between the Jenkins Controller and an Agent?

Answer: *The Controller (master) is the main Jenkins server — it hosts the UI, manages jobs, and orchestrates. Agents (nodes) are worker machines where the actual build steps execute. This separation scales workloads.*

Q4. Why should you use the Credentials plugin instead of hardcoding passwords in Jenkinsfiles?

Answer: *Hardcoded passwords are visible in SCM history, logs, and to anyone with repo access. The Credentials plugin stores secrets encrypted in Jenkins and injects them as environment variables at runtime.*

Q5. What port does Jenkins use for the web UI and what port is used for agent connections?

Answer: *8080 for the web UI (configurable), 50000 for JNLP agent connections (also configurable).*

3

Jenkins Jobs — Freestyle & Pipeline

Understanding the two core job types

3.1 Job Types in Jenkins

Job Type	Description	Use When
Freestyle Project	Classic GUI-based job. Click and configure.	Simple tasks, legacy, quick one-offs
Pipeline	Code-defined pipeline via Jenkinsfile.	ANY real project — industry standard
Multi-branch Pipeline	Auto-discovers branches and creates jobs.	Feature branch workflows, PRs
Organization Folder	Scans entire GitHub org for Jenkinsfiles.	Large teams, many repos
Maven Project	Specialized job for Maven builds.	Pure Maven Java projects
Folder	Groups jobs — not a build job itself.	Organizing large Jenkins setups

3.2 Freestyle Jobs — Understanding the Structure

Freestyle jobs are configured via the Jenkins UI. While not the modern standard, understanding them helps you grasp pipeline concepts:

Freestyle Job Sections:

1. General

- Job description
- Discard old builds (keep last N)
- GitHub project URL

2. Source Code Management

- Git repository URL
- Credentials (username/password or SSH key)

- Branch to build (*/main, */develop)

3. Build Triggers

- Poll SCM: H/5 * * * * (every 5 minutes)
- GitHub hook trigger (webhook)
- Build periodically: 0 2 * * * (2 AM daily)
- Trigger after another job completes

4. Build Environment

- Delete workspace before build
- Inject environment variables
- Set build timeout

5. Build Steps

- Execute shell: mvn clean test
- Invoke Maven goals
- Execute Windows batch command

6. Post-build Actions

- Archive artifacts (target/*.jar)
- Publish test results
- Send email notification
- Trigger other jobs

3.3 Pipeline Jobs — The Industry Standard

Pipeline jobs are defined in a **Jenkinsfile** — a text file stored in your source code repository. This is called **Pipeline as Code** and is the industry best practice.

REMEMBER

Pipeline as Code Benefits: Version controlled, peer-reviewed, reproducible, auditable, templatable, diff-able, and shareable across teams. ALWAYS prefer pipeline over freestyle for any real project.

3.4 Build Triggers — Deep Dive

Trigger Type	Config	When to Use
Webhook (Push trigger)	GitHub sends HTTP POST on push	Best — instant, no polling overhead
Poll SCM	H/5 * * * *	When webhooks aren't possible
Scheduled	0 2 * * *	Nightly builds, reports, cleanup
Manual	Build Now button	On-demand, hotfix releases
Upstream	After job X completes	Job chaining, sequential pipelines
API trigger	POST /job/name/build	External systems trigger builds

Cron Syntax Reference

Cron syntax for Jenkins (H = Hash — avoids all jobs running at same time)

H/5 * * * * # Every ~5 minutes (hash prevents stampede)

H H * * * # Once per day at a random hour

0 2 * * 1-5 # At 2 AM, Monday through Friday

0 0 * * 0 # Every Sunday midnight

H H(8-16) * * 1-5 # Randomly between 8 AM-4 PM on weekdays

Format: MINUTE HOUR DAY-OF-MONTH MONTH DAY-OF-WEEK

Topic 3 — Practice Questions

Practice Questions & Answers

Q1. What is the main advantage of a Pipeline job over a Freestyle job?

Answer: Pipeline jobs are defined as code (Jenkinsfile) in source control. This means they're version-controlled, auditable, peer-reviewable, and reproducible. Freestyle jobs are configured via UI clicks that can't be easily tracked or shared.

Q2. What does 'H/15 * * * *' mean in Jenkins cron syntax?

Answer: Poll for changes approximately every 15 minutes. The H (Hash) spreads the load across Jenkins by hashing the job name to pick a random minute offset, preventing all jobs from polling at exactly the same time.

Q3. When would you choose a Multi-branch Pipeline over a regular Pipeline?

Answer: When you use Git branching workflows (GitFlow, GitHub Flow). Multi-branch Pipeline automatically creates a Jenkins job for every branch that has a Jenkinsfile, allowing you to run CI on feature branches and PRs automatically.

Q4. What happens to the Jenkins workspace between builds by default?

Answer: The workspace is reused between builds — files from the previous build remain. You must explicitly clean the workspace using 'cleanWs()' or 'deleteDir()' if you need a fresh start every build.

Q5. A build ran 2 hours ago and failed. How do you see what happened?

Answer: Go to the job, click on the failed build number, then 'Console Output'. You'll see every command that ran and where/why it failed. You can also check the 'Test Results' if tests ran.

4

Jenkinsfile & Declarative Pipeline Syntax

Writing pipelines as code — the most important skill

4.1 Declarative vs Scripted Pipeline

Type	Syntax	Best For
Declarative	Structured, defined sections (pipeline{})	99% of use cases — industry standard
Scripted	Full Groovy code (node{} blocks)	Complex logic, legacy pipelines

INFO

Always use Declarative Pipeline for new projects. It's simpler, more readable, has built-in error handling, and is the direction Jenkins is actively developing. Scripted pipeline is Groovy — powerful but complex.

4.2 Declarative Pipeline — Full Anatomy

```
# Complete Declarative Jenkinsfile
```

```
pipeline {  
    // Where does this run?  
    agent any  
  
    // Tools to install/use  
    tools {  
        maven 'Maven-3.9'  
        jdk 'JDK-17'  
    }  
  
    // Environment variables  
    environment {
```

```
APP_NAME = 'myapp'

DOCKER_REG = 'myregistry.io'

// Retrieve secret from Jenkins credentials

DOCKER_PASS = credentials('docker-hub-password')

}

// What triggers this pipeline

triggers {
pollSCM('H/5 * * * *')
// or: githubPush()
}

// Build options

options {
timeout(time: 30, unit: 'MINUTES')

buildDiscarder(logRotator(numToKeepStr: '10'))

disableConcurrentBuilds()

timestamps()
}

// The actual build stages

stages {
stage('Checkout') {
steps {
git branch: 'main',
url: 'https://github.com/org/repo.git',
credentialsId: 'github-token'
}
}
}
```

```
stage('Build') {
  steps {
    sh 'mvn clean package -DskipTests'
  }
}

stage('Test') {
  steps {
    sh 'mvn test'
  }
  post {
    always {
      junit 'target/surefire-reports/*.xml'
    }
  }
}

stage('Docker Build & Push') {
  steps {
    script {
      def image = docker.build("${DOCKER_REG}/${APP_NAME}:${BUILD_NUMBER}")
      docker.withRegistry('https://myregistry.io', 'docker-credentials') {
        image.push()
        image.push('latest')
      }
    }
  }
}
```

```
stage('Deploy to Staging') {
  when {
    branch 'main'
  }
  steps {
    sh 'kubectl set image deployment/myapp myapp=${DOCKER_REG}/${APP_NAME}:${BUILD_NUMBER}'
  }
}

// Post-build actions
post {
  success {
    slackSend channel: '#deployments', message: "Build ${BUILD_NUMBER} SUCCESS"
  }
  failure {
    mail to: 'team@company.com',
    subject: "Build FAILED: ${JOB_NAME} #${BUILD_NUMBER}",
    body: "Check: ${BUILD_URL}"
  }
  always {
    cleanWs() // Clean workspace after every build
  }
}
}
```

4.3 Agent Directives


```
// Run on any available agent
agent any

// Run on a specific labeled agent
agent { label 'linux-docker' }

// Run inside a Docker container
agent {
  docker {
    image 'node:18-alpine'
    args '-v /tmp:/tmp'
  }
}

// No global agent – each stage defines its own
agent none

// Then in each stage:
stage('Build') {
  agent { label 'build-server' }
  steps { ... }
}

// Run on Kubernetes pod
agent {
  kubernetes {
    yaml '''
spec:
  containers:
  - name: maven
```

```
image: maven:3.9-eclipse-temurin-17
...
}
}
```

4.4 The 'when' Directive — Conditional Stages

```
stage('Deploy to Production') {
  when {
    // Only run when on main branch
    branch 'main'
  }
  steps { ... }
}

stage('Run Integration Tests') {
  when {
    // Only when environment variable is set
    environment name: 'RUN_INTEGRATION', value: 'true'
  }
  steps { ... }
}

stage('Release') {
  when {
    // Combine conditions
    allOf {
      branch 'main'
```

```
not { changeRequest() } // Not a PR

}

}

steps { ... }

}
```

4.5 Parallel Stages — Speed Up Your Pipeline

```
stage('Tests') {
    parallel {
        stage('Unit Tests') {
            steps {
                sh 'mvn test -Dgroups=unit'
            }
        }
        stage('Integration Tests') {
            steps {
                sh 'mvn test -Dgroups=integration'
            }
        }
        stage('SonarQube Analysis') {
            steps {
                sh 'mvn sonar:sonar'
            }
        }
    }
}
```

```
// Result: All 3 run simultaneously instead of sequentially  
  
// Saves time proportional to the slowest parallel stage
```

4.6 Built-in Jenkins Environment Variables

Variable	Contains
BUILD_NUMBER	Current build number (1, 2, 3, ...)
BUILD_ID	Build ID (timestamp-based)
BUILD_URL	Full URL to this build's page
JOB_NAME	Name of the job
WORKSPACE	Absolute path to the workspace directory
GIT_BRANCH	Branch being built (e.g. origin/main)
GIT_COMMIT	Full SHA of the commit being built
GIT_PREVIOUS_COMMIT	Previous commit SHA
JENKINS_URL	Base URL of the Jenkins server
NODE_NAME	Name of the agent running the build

Topic 4 — Practice Questions

Practice Questions & Answers

Q1. What is the difference between 'sh' and 'bat' steps in a Jenkinsfile?

Answer: *'sh' runs shell commands on Linux/Mac agents. 'bat' runs batch commands on Windows agents. For cross-platform pipelines, use the 'isUnix()' helper or abstract the command into a script file.*

Q2. What does 'cleanWs()' do and when should you call it?

Answer: *cleanWs() deletes the entire Jenkins workspace for that job after the build. Call it in the post { always { } } block to prevent disk space accumulation from build artifacts accumulating across hundreds of builds.*

Q3. How do you safely use a password in a Jenkinsfile without hardcoding it?

Answer: *Store it in Jenkins Credentials (Manage Jenkins > Credentials). In the pipeline, use: environment { MY_PASS = credentials('credential-id') }. Jenkins masks the value in all logs automatically.*

Q4. A stage is taking 20 minutes and blocking everything. How do you speed it up?

Answer: *Split the slow stage into sub-tasks and run them in parallel using the parallel{} block. For example, unit tests, integration tests, and code analysis can all run simultaneously.*

Q5. What does 'agent none' at the top level mean and when do you use it?

Answer: *It means no default agent — each stage must declare its own agent. Use it when different stages need different environments, e.g. build on a Java agent, deploy from a different agent with kubectl access.*

Q6. Write the 'when' condition to only run a 'Deploy' stage if the branch is 'main' AND it's not a pull request.

Answer: *when { allOf { branch 'main'; not { changeRequest() } } }*

5

Source Control Integration — Git & GitHub

Connecting Jenkins to your code repositories

5.1 Connecting Jenkins to GitHub

```
# Method 1: HTTPS with Personal Access Token (PAT)

1. GitHub > Settings > Developer Settings > Personal Access Tokens

2. Generate token with: repo, read:user, admin:repo_hook

3. Jenkins > Manage Jenkins > Credentials > Add Credential

Kind: Username with password

Username: your-github-username

Password: your-PAT-token

ID: github-token

# Method 2: SSH Key (recommended for production)

1. Generate SSH key pair on Jenkins server:

ssh-keygen -t ed25519 -C 'jenkins@yourserver.com'

2. Add public key to GitHub > Settings > SSH Keys

3. Add private key to Jenkins > Credentials

Kind: SSH Username with private key

ID: github-ssh-key

# In Jenkinsfile:

stage('Checkout') {
  steps {
    git branch: 'main',
    credentialsId: 'github-token',
```

```
url: 'https://github.com/username/repo.git'

}
```

5.2 Setting Up GitHub Webhooks

Webhooks are the preferred way to trigger Jenkins builds. Instead of Jenkins polling GitHub every X minutes, GitHub pushes a notification to Jenkins instantly on every push.

Setup Steps:

1. Your Jenkins must be publicly accessible (or use ngrok for local dev)

2. GitHub Repository > Settings > Webhooks > Add webhook

Payload URL: `http://YOUR-JENKINS-URL/github-webhook/`

Content type: `application/json`

Secret: (optional but recommended)

Events: Just the push event (or select specific events)

3. In Jenkins Pipeline, set trigger:

```
triggers {
  githubPush()
}
```

4. Test: Push a commit to the repo and watch Jenkins auto-trigger

For local development, use ngrok to expose Jenkins:

```
ngrok http 8080
```

Use the ngrok URL as your Payload URL in GitHub

5.3 Multi-Branch Pipeline with GitHub

```
# Multi-branch Pipeline automatically creates a Jenkins job
# for EVERY branch that has a Jenkinsfile.

# Setup in Jenkins UI:
New Item > Multi-branch Pipeline
Branch Sources > Add source > GitHub
Repository HTTPS URL: https://github.com/org/repo
Credentials: github-token
Scan Repository Triggers: Periodically (H/5 * * * *)

# Jenkinsfile can behave differently per branch:
pipeline {
  agent any

  stages {
    stage('Build') { steps { sh 'mvn package' } }
    stage('Deploy to Staging') {
      when { branch 'develop' }
      steps { sh './deploy.sh staging' }
    }
    stage('Deploy to Production') {
      when { branch 'main' }
      steps {
        input 'Deploy to production?' // Manual approval
        sh './deploy.sh production'
      }
    }
  }
}
```


Topic 5 — Practice Questions

Practice Questions & Answers

Q1. What is the security advantage of using SSH keys over HTTPS tokens for GitHub auth?

Answer: *SSH keys never leave the server and can't be phished. Tokens can be accidentally logged or exposed in URLs. SSH keys also support passphrase protection and can be restricted to specific permissions per key.*

Q2. What is the difference between polling SCM and a webhook trigger?

Answer: *Polling: Jenkins checks GitHub every X minutes for changes — resource-intensive, delayed feedback. Webhook: GitHub instantly notifies Jenkins when a push happens — zero delay, no wasted polling requests.*

Q3. Why does a Multi-branch Pipeline need to 'scan' the repository?

Answer: *To discover all branches that have a Jenkinsfile. When a new branch is created or deleted, the scan updates Jenkins' job list to match. Without scanning, new branches would never get a CI job.*

Q4. How do you add a manual approval gate before deploying to production?

Answer: *Use the 'input' step: input message: 'Deploy to production?', ok: 'Yes, deploy!'. This pauses the pipeline and waits for a human to click Proceed or Abort in the Jenkins UI.*

Q5. A webhook is configured but builds aren't triggering. What do you check first?

Answer: *1) Is Jenkins accessible from the internet? 2) Check GitHub Webhook settings for the last delivery and its response code. 3) Check Jenkins system log for incoming requests. 4) Verify the trigger in Jenkinsfile is set to githubPush(). 5) Check Jenkins GitHub plugin configuration.*

6

Build Tools — Maven, Gradle & npm

Compiling, testing and packaging your applications

6.1 Maven — Java Build Tool

```
# Key Maven commands used in Jenkins pipelines

mvn clean # Delete target/ folder (fresh start)

mvn compile # Compile source code

mvn test # Compile + run unit tests

mvn package # Compile + test + create JAR/WAR

mvn verify # Run integration tests

mvn install # Package + install to local ~/.m2 cache

mvn deploy # Publish to remote artifact repository

# Useful flags:

mvn package -DskipTests # Build without running tests (faster)

mvn package -Pprod # Activate 'prod' profile

mvn -B # Batch mode (less output, better for CI)

mvn -q package # Quiet mode

# In Jenkinsfile:

tools { maven 'Maven-3.9' }

...

sh 'mvn -B clean package'
```

6.2 Gradle — Modern Java Build Tool

```
# Key Gradle commands

./gradlew build # Full build (compile + test + package)

./gradlew test # Run tests only

./gradlew clean build # Fresh build

./gradlew bootJar # Spring Boot executable JAR

./gradlew dependencies # Show dependency tree

# ALWAYS use the Gradle wrapper (gradlew) in CI – not system Gradle

# The wrapper ensures everyone uses the same Gradle version

# In Jenkinsfile:
stage('Build') {
  steps {
    sh 'chmod +x ./gradlew'

    sh './gradlew clean build'
  }

  post {
    always {
      junit 'build/test-results/**/*.xml'
    }
  }
}
```

6.3 npm — Node.js Build Tool

```
# Key npm commands in CI

npm ci # ALWAYS use this in CI (not 'npm install')

# Uses package-lock.json for reproducible builds
```

```
npm test # Run tests

npm run build # Build production bundle

npm run lint # Lint code

npm audit # Check for security vulnerabilities

npm audit fix # Fix vulnerabilities automatically


# In Jenkinsfile:

pipeline {
  agent {
    docker { image 'node:18-alpine' }
  }
  stages {
    stage('Install') {
      steps { sh 'npm ci' }
    }
    stage('Test') {
      steps { sh 'npm test -- --coverage' }
    }
    stage('Build') {
      steps { sh 'npm run build' }
    }
  }
}
```

6.4 Artifact Management — Archiving Build Outputs

```
// Archive artifacts for download from Jenkins UI

post {
```

```
success {  
  
  archiveArtifacts artifacts: 'target/*.jar', fingerprint: true  
  archiveArtifacts artifacts: 'dist/**/*.jar'  
  
}  
  
// For enterprise: use Nexus or JFrog Artifactory  
// Publish Maven artifact to Nexus:  
sh 'mvn deploy -DaltDeploymentRepository=nexus::default::http://nexus:8081/repository/releases'  
  
// Stash/unstash between stages (within same build)  
stage('Build') {  
  steps {  
    sh 'mvn package -DskipTests'  
    stash includes: 'target/*.jar', name: 'app-jar'  
  }  
}  
  
stage('Deploy') {  
  agent { label 'deploy-server' }  
  steps {  
    unstash 'app-jar'  
    sh 'java -jar target/app.jar'  
  }  
}
```

Topic 6 — Practice Questions

Practice Questions & Answers

Q1. Why use 'npm ci' instead of 'npm install' in CI pipelines?

Answer: *'npm ci' installs EXACTLY the versions in package-lock.json with zero deviation. 'npm install' may update packages and modify package-lock.json. In CI, reproducible builds are critical — 'npm ci' also fails if package.json and package-lock.json are out of sync.*

Q2. What does 'mvn -B clean package' do and what does -B mean?

Answer: *mvn -B clean package: -B is batch mode (suppresses download progress bars for cleaner CI logs). clean deletes the previous build output. package compiles code, runs tests, and creates the JAR/WAR file.*

Q3. What is the Gradle wrapper and why must you use it in CI?

Answer: *The Gradle wrapper (gradlew script + wrapper jar) downloads and runs a specific version of Gradle. In CI, this guarantees every build uses the exact same Gradle version regardless of what's installed on the agent — eliminating 'works on my machine' issues.*

Q4. What is artifact fingerprinting in Jenkins?

Answer: *Fingerprinting records the MD5 hash of a build artifact and tracks which builds produced it and which jobs consumed it. This helps audit: 'Which exact version of the JAR is running in production?'*

Q5. When would you use 'stash' and 'unstash' in a Jenkinsfile?

Answer: *When different stages run on different agents (nodes) and need to share files. Stash saves files into Jenkins controller storage. Unstash retrieves them on another agent. Example: build the JAR on a build agent, then unstash it on a deploy agent with cloud access.*

7

Testing, Code Quality & SonarQube

Ensuring your code is correct and maintainable

7.1 Types of Tests in CI/CD

Test Type	What It Tests	Speed	Run in CI?
Unit Tests	Individual functions/classes in isolation	Very fast (ms)	Every build
Integration Tests	Multiple components working together	Medium (seconds)	Every build or daily
End-to-End (E2E)	Full application flow via browser/API	Slow (minutes)	Daily or pre-release
Performance Tests	Response time, throughput, load	Very slow	Weekly or pre-release
Security Tests	OWASP vulnerabilities, CVEs	Medium	Every build (SAST) + weekly

7.2 Publishing Test Results in Jenkins

```
pipeline {
  stages {
    stage('Test') {
      steps {
        sh 'mvn test'
      }
      post {
        always {
          // JUnit XML results (Java/Maven)
          junit 'target/surefire-reports/*.xml'
          junit '**/test-results/**/*.xml'

          // HTML report (any framework)
```

```
publishHTML([
  allowMissing: false,
  reportDir: 'target/site/jacoco',
  reportFiles: 'index.html',
  reportName: 'JaCoCo Coverage Report'
])

// Cobertura coverage (Python)
// cobertura coberturaReportFile: 'coverage.xml'
}
}
}
}
}
```

// After publishing, Jenkins shows:

- // - Test trend chart on job page
- // - Failed test names and stack traces
- // - Test history across builds

7.3 SonarQube — Code Quality Analysis

```
# Step 1: Run SonarQube (via Docker)

docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community

# Visit http://localhost:9000 (admin/admin)

# Step 2: Create a project in SonarQube UI

# Generate a project token

# Step 3: Configure Jenkins
```



```
# Manage Jenkins > Configure System > SonarQube servers

# Name: SonarQube

# URL: http://sonarqube:9000

# Token: (add as Jenkins secret credential)

# Step 4: In Jenkinsfile

pipeline {
  tools { maven 'Maven-3.9' }
  stages {
    stage('SonarQube Analysis') {
      steps {
        withSonarQubeEnv('SonarQube') {
          sh 'mvn sonar:sonar -Dsonar.projectKey=myapp'
        }
      }
    }
    stage('Quality Gate') {
      steps {
        // Wait for SonarQube to process and return result
        timeout(time: 5, unit: 'MINUTES') {
          waitForQualityGate abortPipeline: true
        }
      }
    }
  }
}
```

// If Quality Gate FAILS -> Build fails -> Nothing gets deployed

```
// This enforces code quality as a deployment gate
```

7.4 SonarQube Quality Gate Metrics

Metric	Good Threshold	What It Measures
Coverage	> 80%	% of code covered by tests
Duplications	< 3%	Copy-paste code (technical debt)
Code Smells	A rating	Maintainability issues
Bugs	0	Definite code errors
Vulnerabilities	0	Security weaknesses
Security Hotspots	Reviewed	Code that might have security issues

Topic 7 — Practice Questions

Practice Questions & Answers

Q1. Why should test results be published with 'always' in the post block, not 'success'?

Answer: *Because when tests FAIL, the build is marked as failure. If you use 'success', the test reports never get published on failed builds — which is exactly when you NEED the reports most to see which tests failed and why.*

Q2. What is a Quality Gate in SonarQube?

Answer: *A Quality Gate is a set of conditions (e.g. coverage > 80%, 0 bugs) that code must meet. If the analysis fails the gate, 'waitForQualityGate abortPipeline: true' will fail the Jenkins build, preventing bad code from being deployed.*

Q3. What is code coverage and what does 80% coverage mean?

Answer: *Code coverage measures what % of your code lines are executed by tests. 80% coverage means 80% of code lines run during tests. It does NOT guarantee correctness — tests could exist but make no assertions.*

Q4. A build passes all tests but you discover a critical bug in production. What went wrong?

Answer: *The critical path wasn't tested. Tests existed but didn't cover that scenario. This could be: missing edge case tests, integration not tested end-to-end, or the production environment has different config that wasn't mocked. Answer: improve integration/E2E test coverage.*

Q5. How is SAST (Static Analysis) different from running tests?

Answer: *Tests execute the code at runtime to verify behavior. SAST (Static Application Security Testing like SonarQube) analyzes the source code WITHOUT running it — finding bugs, security vulnerabilities, and code smells by examining code patterns.*

8

Docker Integration in Jenkins Pipelines

Containerizing your builds and artifacts

8.1 Two Ways to Use Docker in Jenkins

Approach	How	Best For
Docker as build environment	agent { docker { image 'node:18' } }	Running builds in clean containers
Docker as artifact	docker.build() / docker.push()	Building and pushing app images

8.2 Using Docker as Build Agent

```
// Run the entire pipeline inside a Docker container
// Jenkins pulls the image, runs your steps inside it
pipeline {
  agent {
    docker {
      image 'maven:3.9-eclipse-temurin-17'
      // Mount Maven cache to speed up builds
      args '-v $HOME/.m2:/root/.m2'
    }
  }
  stages {
    stage('Build') {
      steps {
        // mvn is available because we're inside the Maven image!
```

```
sh 'mvn clean package'

}

}

}

}

// Per-stage Docker agents:

pipeline {
  agent none

  stages {
    stage('Build Java') {
      agent { docker { image 'maven:3.9-jdk-17' } }
      steps { sh 'mvn package' }
    }
    stage('Build Frontend') {
      agent { docker { image 'node:18-alpine' } }
      steps { sh 'npm ci && npm run build' }
    }
  }
}
```

8.3 Building & Pushing Docker Images

```
pipeline {
  agent any

  environment {
    REGISTRY = 'docker.io'
    IMAGE_NAME = 'myusername/myapp'
```

```
IMAGE_TAG = "${BUILD_NUMBER}-${GIT_COMMIT.take(7)}"

}

stages {

stage('Build Docker Image') {

steps {

script {

// Build the image

dockerImage = docker.build("${IMAGE_NAME}:${IMAGE_TAG}")

}

}

}

stage('Push to Registry') {

steps {

script {

// Authenticate using Jenkins credentials

docker.withRegistry('https://registry.hub.docker.com', 'dockerhub-credentials') {

dockerImage.push() // Push with build number tag

dockerImage.push('latest') // Also tag as latest

}

}

}

}

stage('Clean Up Local Image') {

steps {

// Remove image from Jenkins agent to save disk space

sh "docker rmi ${IMAGE_NAME}:${IMAGE_TAG}"

}
```

```
}  
  
}  
  
}  
  
}
```

8.4 Multi-Stage Dockerfile for CI/CD

```
# Dockerfile with multiple stages – key for efficient CI/CD  
  
# Stage 1: Build (large image with build tools)  
FROM maven:3.9-eclipse-temurin-17 AS builder  
  
WORKDIR /app  
  
COPY pom.xml .  
  
RUN mvn dependency:go-offline -B # Cache dependencies  
  
COPY src ./src  
  
RUN mvn clean package -DskipTests  
  
# Stage 2: Test (run tests against built artifact)  
FROM builder AS tester  
  
RUN mvn test  
  
# Stage 3: Production (minimal image – only the JAR)  
FROM eclipse-temurin:17-jre-alpine AS production  
  
WORKDIR /app  
  
# Copy ONLY the JAR from the builder stage  
COPY --from=builder /app/target/myapp.jar app.jar  
  
EXPOSE 8080  
  
ENTRYPOINT ["java", "-jar", "app.jar"]  
  
# Result: Production image is ~100MB instead of ~500MB
```

```
# No build tools, no source code in production image
```

Topic 8 — Practice Questions

Practice Questions & Answers

Q1. Why mount the Maven cache directory when using Docker as a build agent?

Answer: Without mounting the cache, every build downloads all Maven dependencies from the internet — slow and bandwidth-heavy. Mounting `~/.m2` as a volume means dependencies are cached on the host and reused across builds, cutting build time dramatically.

Q2. What is the benefit of tagging a Docker image with both `BUILD_NUMBER` and 'latest'?

Answer: `BUILD_NUMBER` creates a permanent, unique, traceable tag for that exact build (e.g. `myapp:142`). 'latest' always points to the most recent successful build. Services pulling 'latest' get the newest version, but you can always roll back to any specific build number.

Q3. What is a multi-stage Dockerfile and why is it important for CI/CD?

Answer: A multi-stage Dockerfile uses multiple `FROM` statements. Early stages do heavy build work (Maven, npm). The final stage copies only the built artifact into a minimal base image. Result: small, secure production images without build tools or source code.

Q4. Why shouldn't you hardcode Docker registry credentials in a Jenkinsfile?

Answer: The Jenkinsfile is stored in Git — anyone with repo access sees the credentials. Use Jenkins Credentials store and inject them via `docker.withRegistry('url', 'credential-id')`. Jenkins masks the password in all build logs.

Q5. How do you avoid Docker image layer bloat over many builds?

Answer: 1) Add a stage to remove the local image after pushing. 2) Use `docker system prune` in a scheduled Jenkins job. 3) Use `.dockerignore` to exclude unnecessary files. 4) Use multi-stage builds to keep final images small.

9

Deployment Strategies & Best Practices

Getting code to production safely

9.1 Deployment Strategies Compared

Strategy	How It Works	Downtime	Risk	Rollback
Rolling Update	Replace instances one by one	None	Medium	Slow
Blue/Green	Two identical envs, switch traffic	None	Low	Instant
Canary	Route X% traffic to new version	None	Very Low	Instant
Recreate	Kill all, then deploy new version	Yes	High	Manual
A/B Testing	Route by user segment, not %	None	Low	Instant

9.2 Blue/Green Deployment in Jenkins

```
pipeline {
  agent any
  environment {
    IMAGE = "myapp:${BUILD_NUMBER}"
  }
  stages {
    stage('Build & Push') {
      steps {
        sh "docker build -t ${IMAGE} ."
        sh "docker push ${IMAGE}"
      }
    }
  }
}
```

```
stage('Deploy to Green') {
  steps {
    // Deploy new version to 'green' environment
    sh """
    docker stop myapp-green || true
    docker rm myapp-green || true
    docker run -d --name myapp-green \
    -p 8081:8080 \
    ${IMAGE}
    """
  }
}

stage('Smoke Test Green') {
  steps {
    sh 'sleep 10' // Wait for app to start
    sh 'curl -f http://localhost:8081/health'
  }
}

stage('Switch Traffic') {
  steps {
    input 'Switch production traffic to Green?'
    // Update nginx or load balancer to point to Green
    sh './switch-traffic.sh green'
  }
}

stage('Decommission Blue') {
```

```
steps {  
  sh 'docker stop myapp-blue || true'  
  sh 'docker rm myapp-blue || true'  
}  
}  
}
```

9.3 Notifications — Slack & Email

```
// Slack notifications  
  
post {  
  success {  
    slackSend(  
      channel: '#deployments',  
      color: 'good',  
      message: "SUCCESS: ${JOB_NAME} #${BUILD_NUMBER} | ${BUILD_URL}"  
    )  
  }  
  failure {  
    slackSend(  
      channel: '#alerts',  
      color: 'danger',  
      message: "FAILED: ${JOB_NAME} #${BUILD_NUMBER}\nCheck: ${BUILD_URL}console"  
    )  
  }  
}
```

```
// Email notifications

post {
  failure {
    emailext(
      to: 'team@company.com',
      subject: "Build FAILED: ${JOB_NAME} #${BUILD_NUMBER}",
      body: ""
        Build ${BUILD_NUMBER} failed for ${JOB_NAME}.
        Branch: ${GIT_BRANCH}
        Commit: ${GIT_COMMIT}
        Console: ${BUILD_URL}console
      "",
      attachLog: true
    )
  }
}
```

9.4 Jenkins Best Practices — The Pro Rules

1. ALWAYS store Jenkinsfile in the source repository

Never configure pipelines through the UI only

2. ALWAYS use declarative pipeline over scripted

More readable, better error handling, actively maintained

3. NEVER hardcode credentials, passwords, or tokens

Use Jenkins Credentials + environment { MY_VAR = credentials('id') }

4. ALWAYS set timeouts

```
options { timeout(time: 30, unit: 'MINUTES') }
```

Prevents runaway builds from consuming agents forever

5. ALWAYS clean the workspace after builds

```
post { always { cleanWs() } }
```

6. USE specific image tags, never 'latest'

```
docker.build('myapp:1.2.3') not docker.build('myapp:latest')
```

7. RUN parallel stages wherever possible

Unit tests + linting + security scan = huge time savings

8. KEEP pipeline logic in scripts, not Jenkinsfiles

Jenkinsfile calls: `sh './scripts/build.sh'`

Not: 50 lines of shell in the Jenkinsfile

9. USE 'when' conditions to skip irrelevant stages

Don't deploy to prod on every feature branch build

10. MONITOR Jenkins itself

Set up disk space alerts, plugin update notifications

Keep Jenkins and plugins updated for security patches

Topic 9 — Practice Questions

Practice Questions & Answers

Q1. What is the key advantage of Blue/Green deployment over a Rolling update?

Answer: *In Blue/Green, the old version (Blue) keeps running during the entire deployment. If anything goes wrong with Green, you switch traffic back to Blue instantly with zero downtime. Rolling updates modify running instances — rollback is slow and complex.*

Q2. What is a canary deployment and when would you use it?

Answer: *Route a small % of real traffic (e.g. 5%) to the new version while 95% goes to the old. Monitor error rates and performance. If metrics are good, gradually increase to 100%. Use it for high-risk changes in production where you want real-world validation with minimal blast radius.*

Q3. Why should you never put sensitive logic in a Jenkinsfile?

Answer: *Jenkinsfiles are stored in Git — visible to everyone with repo access. Logic means business logic, credentials, IP addresses, internal system details. Put complex logic in scripts stored separately, or in shared libraries. Jenkinsfiles should be thin orchestration wrappers.*

Q4. What happens if you don't set a pipeline timeout and a build hangs?

Answer: *The build runs forever, consuming an agent. All other jobs waiting for that agent are blocked. Eventually you run out of available agents and your entire CI system stalls. Jenkins won't kill hung builds automatically without a timeout.*

Q5. How do you roll back a deployment if a canary shows increased error rates?

Answer: *Route 100% of traffic back to the stable version (the canary routes 0%). Stop and remove the canary instances. Investigate the issue in a safe environment. Fix, re-test, then gradually re-deploy.*

Resume Projects — Build Your Portfolio

These 6 projects are specifically designed to look impressive on your resume and demonstrate real-world Jenkins competence to interviewers. Complete them in order — each builds on the last.

HANDS-ON PROJECT

PROJECT 1: Personal Portfolio Auto-Deploy Pipeline

Difficulty: Beginner | **Time:** 3-4 hours | **Resume Value:** Shows CI/CD basics

What you build: A Jenkins pipeline that automatically deploys a static website (HTML/CSS/JS portfolio page) to an Nginx web server whenever you push to GitHub.

Tech Stack: Git, GitHub, Jenkins, Nginx, Docker

Pipeline Stages:

1. Checkout -> Pull code from GitHub
2. Validate -> Check HTML with tidy or htmlhint
3. Build -> Copy files to deploy directory
4. Deploy -> Restart Nginx container with new files
5. Notify -> Send email on success/failure

Jenkinsfile skeleton:

```
pipeline {
  agent any

  triggers { githubPush() }

  stages {

    stage('Checkout') { steps { checkout scm } }

    stage('Validate') { steps { sh 'npx htmlhint src/**/*.html' } }

    stage('Deploy') {

      steps {
```

```
sh 'docker cp ./src/. nginx-container:/usr/share/nginx/html/'

sh 'docker exec nginx-container nginx -s reload'

}

}

}

post {

  success { mail to: 'you@email.com', subject: 'Portfolio deployed!', body: BUILD_URL }

  failure { mail to: 'you@email.com', subject: 'Deploy FAILED', body: BUILD_URL }

}

}
```

Resume line: 'Implemented automated deployment pipeline for personal portfolio using Jenkins with GitHub webhook triggers and zero-downtime Nginx serving.'

HANDS-ON PROJECT

PROJECT 2: Java Spring Boot — Full CI/CD Pipeline

Difficulty: Intermediate | **Time:** 6-8 hours | **Resume Value:** High — most Java jobs require this

What you build: A complete CI/CD pipeline for a Spring Boot REST API including build, test, code quality analysis, Docker image build, and deployment.

Tech Stack: Git, Jenkins, Maven, JUnit, SonarQube, Docker, Docker Hub

Pipeline Stages:

1. Checkout -> Git clone from GitHub
2. Build -> mvn clean package -DskipTests
3. Unit Tests -> mvn test + publish JUnit results
4. Code Quality -> SonarQube analysis + Quality Gate check
5. Docker Build -> Build image with build number tag
6. Docker Push -> Push to Docker Hub

7. Deploy -> docker run on target server

8. Smoke Test -> curl /actuator/health

9. Notify -> Slack notification

Key Jenkinsfile features to implement:

- environment { SONAR_TOKEN = credentials('sonar-token') }
- parallel { stage('Test') {...} stage('OWASP Scan') {...} }
- when { branch 'main' } on deploy stage
- post { always { junit '...'; cleanWs() } }

Resume line: 'Built end-to-end CI/CD pipeline for Java Spring Boot microservice including automated testing, SonarQube quality gates, Docker containerization, and automated deployment to staging environment.'

HANDS-ON PROJECT

PROJECT 3: Node.js App with Docker Build & Registry Push

Difficulty: Intermediate | **Time:** 5-6 hours | **Resume Value:** High — Docker + CI is in demand

Tech Stack: Node.js, Jest, Docker, Docker Hub, Jenkins

Key implementations:

- Use Docker as build agent: agent { docker { image 'node:18-alpine' } }
- npm ci (not npm install) for reproducible builds
- Jest unit tests with coverage report published to Jenkins
- Multi-stage Dockerfile for minimal production image
- Image tagged as both :BUILD_NUMBER and :latest
- Security scan with 'npm audit'
- Automatic rollback on failed smoke test

Bonus challenges:

- Add ESLint stage that fails the build on errors
- Add a 'Deploy to Staging' stage with docker compose
- Set up Multi-branch Pipeline so feature/* branches get CI but only 'main' branch deploys

HANDS-ON PROJECT

PROJECT 4: Multi-Branch Pipeline with Feature Branch Strategy**Difficulty:** Advanced | **Time:** 6-8 hours | **Resume Value:** Shows Git workflow mastery

Tech Stack: Git (GitFlow), Jenkins Multi-branch Pipeline, Docker

What to build:

- Multi-branch pipeline scanning a GitHub repo with multiple branches
- Jenkinsfile that behaves differently per branch:

```
pipeline {
  agent any
  stages {
    stage('CI') { /* Always runs: build + test + quality */ }

    stage('Deploy Dev') {
      when { branch 'develop' }
      steps { sh './deploy.sh dev' }
    }

    stage('Deploy Staging') {
      when { branch 'release/*' }
      steps { sh './deploy.sh staging' }
    }

    stage('Deploy Production') {
      when { branch 'main' }
      steps {
        input message: 'Deploy to production?', ok: 'Release!'
        sh './deploy.sh production'
      }
    }
  }
}
```

```
}  
  
}  
  
}  
  
}
```

Resume line: 'Implemented GitFlow-based multi-branch CI/CD strategy
with branch-specific deployment to dev/staging/production environments
including manual approval gates for production releases.'

HANDS-ON PROJECT

PROJECT 5: Blue/Green Deployment Pipeline

Difficulty: Advanced | **Time:** 8-10 hours | **Resume Value:** Very High — advanced deployment strategy

Tech Stack: Docker, Nginx (load balancer), Jenkins

Pipeline Stages:

1. Build & Push new Docker image
2. Determine current live environment (Blue or Green)
3. Deploy to the INACTIVE environment
4. Run smoke tests against inactive environment
5. Manual approval: 'Switch traffic to new version?'
6. Update Nginx upstream to point to new environment
7. Verify new environment is receiving traffic
8. Keep old environment running for 10 minutes (hot standby)
9. Decommission old environment

Key implementation:

```
script {  
  
def currentEnv = sh(script: 'cat /etc/current-env', returnStdout: true).trim()
```

```
def newEnv = currentEnv == 'blue' ? 'green' : 'blue'

env.DEPLOY_ENV = newEnv

}
```

Resume line: 'Designed and implemented blue/green zero-downtime deployment pipeline with automated smoke testing and instant rollback capability, reducing deployment risk by 90%.'

PROJECT 6 (CAPSTONE): End-to-End DevOps Pipeline — Resume Star Project

Difficulty: Expert | **Time:** 2-3 days | **Resume Value:** Maximum — talk about this in every interview

What you build: A complete, production-grade DevOps pipeline combining EVERYTHING from this course into one impressive, documented project.

Full Architecture:

Application: Microservice with REST API (choose: Java Spring Boot OR Node.js)

Pipeline Flow:

Developer push -> GitHub webhook -> Jenkins triggered

|

[STAGE 1: Code Quality] - SonarQube + Quality Gate

|

[STAGE 2: Security Scan] - OWASP Dependency Check

|

[STAGE 3: Parallel Tests]

+> Unit Tests (Maven/Jest) -> Publish JUnit results

+> Integration Tests -> Docker compose test env

|

[STAGE 4: Build Artifact] - JAR/WAR or npm build

|

[STAGE 5: Docker Build] - Multi-stage Dockerfile

|

[STAGE 6: Docker Push] - Tag with :BUILD_NUMBER and :latest

|

[STAGE 7: Deploy Dev] - Always (on all branches)

|

```
[STAGE 8: Smoke Test] - curl /health on dev  
| (auto-rollback if fails)  
[STAGE 9: Manual Gate] - 'Release to Production?' input  
|  
[STAGE 10: Blue/Green Prod Deploy] - Zero-downtime release  
|  
[STAGE 11: Notify] - Slack success/failure message
```

Infrastructure requirements (all Docker Compose):

- Jenkins controller
- SonarQube + PostgreSQL
- Docker Registry (local)
- Nginx (load balancer for Blue/Green)
- Target application servers (Blue + Green)

Documentation requirements (make this shine on GitHub):

- README with architecture diagram
- Screenshot of successful pipeline run
- Screenshot of SonarQube dashboard
- Explanation of each stage
- How to reproduce the setup

Resume bullet points:

- 'Architected a 10-stage end-to-end CI/CD pipeline integrating Jenkins, SonarQube, OWASP, Docker, and blue/green deployment strategy'
- 'Reduced deployment time by 70% and eliminated manual deployment errors through fully automated pipeline with quality gates and rollback'
- 'Implemented zero-downtime blue/green deployment with automated smoke testing and instant rollback capability'

A

Cheat Sheet & Quick Reference

Everything at your fingertips

Essential Jenkins CLI Commands

```
# Jenkins CLI via Java

java -jar jenkins-cli.jar -s http://localhost:8080 -auth user:token help

# Common CLI commands

java -jar jenkins-cli.jar -s http://localhost:8080 list-jobs
java -jar jenkins-cli.jar -s http://localhost:8080 build JOB_NAME
java -jar jenkins-cli.jar -s http://localhost:8080 build JOB_NAME -p PARAM=value
java -jar jenkins-cli.jar -s http://localhost:8080 delete-job JOB_NAME
java -jar jenkins-cli.jar -s http://localhost:8080 reload-configuration
java -jar jenkins-cli.jar -s http://localhost:8080 install-plugin PLUGIN_NAME

# Jenkins REST API

curl -X POST http://user:token@localhost:8080/job/JOB_NAME/build
curl http://user:token@localhost:8080/job/JOB_NAME/lastBuild/api/json
curl http://user:token@localhost:8080/api/json?pretty=true

# Docker commands for Jenkins management

docker logs jenkins # View Jenkins logs
docker exec -it jenkins bash # Shell into Jenkins container
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
docker restart jenkins # Restart Jenkins
```

Declarative Pipeline Quick Reference

Directive	Location	Purpose
agent	Top-level or stage	Where to run
tools	Top-level or stage	Install tools (Maven, JDK, Node)
environment	Top-level or stage	Set env vars / inject credentials
options	Top-level	Timeout, log rotation, concurrency
parameters	Top-level	User-input params (string, choice, bool)
triggers	Top-level	What starts the pipeline
stages	Top-level	Container for all stage blocks
stage	Inside stages	A named phase of the pipeline
steps	Inside stage	The actual commands to run
parallel	Inside stage	Run multiple stages simultaneously
when	Inside stage	Conditional execution
input	Inside stage	Pause for human approval
post	Top-level or stage	always/success/failure/cleanup actions
script	Inside steps	Run Groovy code block

Credentials Types Reference

Credential Kind	Use For	Access in Pipeline
Username with password	Git HTTPS, Docker registry, APIs	credentials('id') -> USER:PASS
SSH Username with key	Git SSH, server access	ssh agent plugin
Secret text	API tokens, simple passwords	credentials('id') -> text
Secret file	kubeconfig, certificates, .env	credentials('id') -> file path
Certificate	PKCS#12 keystores	credentials('id') -> keystore

Interview Questions — Top 10

Practice Questions & Answers

Q1. Explain the Jenkins pipeline from code commit to production in simple terms.

Answer: *Developer pushes code -> GitHub webhook triggers Jenkins -> Pipeline runs: checkout code, build artifact, run tests, scan for quality, build Docker image, push to registry, deploy to staging, run smoke test, request approval, deploy to production, send notification.*

Q2. What is a shared library in Jenkins and why is it useful?

Answer: *A shared library is a collection of reusable Groovy code stored in a separate Git repo that multiple Jenkinsfiles can import. Instead of copying the same deploy() function into 20 Jenkinsfiles, define it once in a shared library. Updates apply to all pipelines automatically.*

Q3. How do you handle a pipeline that takes too long?

Answer: *1) Identify slow stages using the Blue Ocean stage timing. 2) Run independent stages in parallel. 3) Cache dependencies (Maven ~/.m2, npm node_modules via volume mounts). 4) Use faster agents. 5) Run expensive tests only on main branch, not every feature branch.*

Q4. What is Jenkins Configuration as Code (JCasC)?

Answer: *JCasC lets you define all Jenkins configuration (system settings, credentials, plugins, jobs) in a YAML file. This makes Jenkins setup reproducible, version-controlled, and automated — you can rebuild the entire Jenkins server from a single YAML file.*

Q5. How do you prevent sensitive data from appearing in Jenkins logs?

Answer: *Store secrets in Jenkins Credentials. Inject via environment { SECRET = credentials('id') }. Jenkins automatically masks the value in all console output. Never echo or print credential variables. Use 'set +x' in shell scripts to prevent command echo.*

Q6. What would you do if a Jenkins build is stuck and not completing?

Answer: *1) Check Console Output for the last output line. 2) If hung on user input — abort or respond. 3) If SSH/network issue — check agent connectivity. 4) If resource issue — check disk/memory on agent. 5) Kill the build from UI. 6) Investigate root cause and add timeout to prevent recurrence.*

REMEMBER**You Are Now Jenkins CI/CD Ready! Congratulations!**

You have studied all 9 topics, answered 50+ practice questions, and built 6 portfolio projects. You are now prepared to:

- Design CI/CD pipelines from scratch for any technology stack
- Write production-quality Jenkinsfiles with parallel stages, quality gates, and notifications
- Implement zero-downtime deployment strategies (Blue/Green, Canary)
- Secure Jenkins with proper credentials management and network hardening
- Confidently answer Jenkins questions in DevOps interviews

Next Steps: *Learn Kubernetes (K8s) to deploy your Jenkins pipelines to cloud-scale infrastructure. Your Jenkinsfile skills transfer directly to Kubernetes-based CI/CD.*